

Design Rationale Req5

Design1

Design 1 constructs the Mysterio Store system by combining several components, including Interior, Session, Directory, BuyAction, EnterAction, and LeaveAction, which work together to manage player interactions within the store. The LifecycleInterface defines how each store instance is created, opened, and closed, while ExitPortal and DimensionalSiteGround handle teleportation between the Earth map and the store dimension. All modules are coordinated through Session and Directory to ensure smooth entry, interaction, and exit processes for the player.

Pros	Cons
Follows OOP and SOLID principles by assigning clear responsibilities to each component.	Violates the KISS principle — the design is overly fragmented with too many small classes and interfaces.
	High coupling between modules (e.g., Session, Directory, and LifecycleInterface) makes debugging and integration harder.
	Increases implementation complexity and setup cost since many objects must be instantiated and coordinated.

Design2

Design 2 adopts a modular and object-oriented approach where each class has a single, clearly defined responsibility.

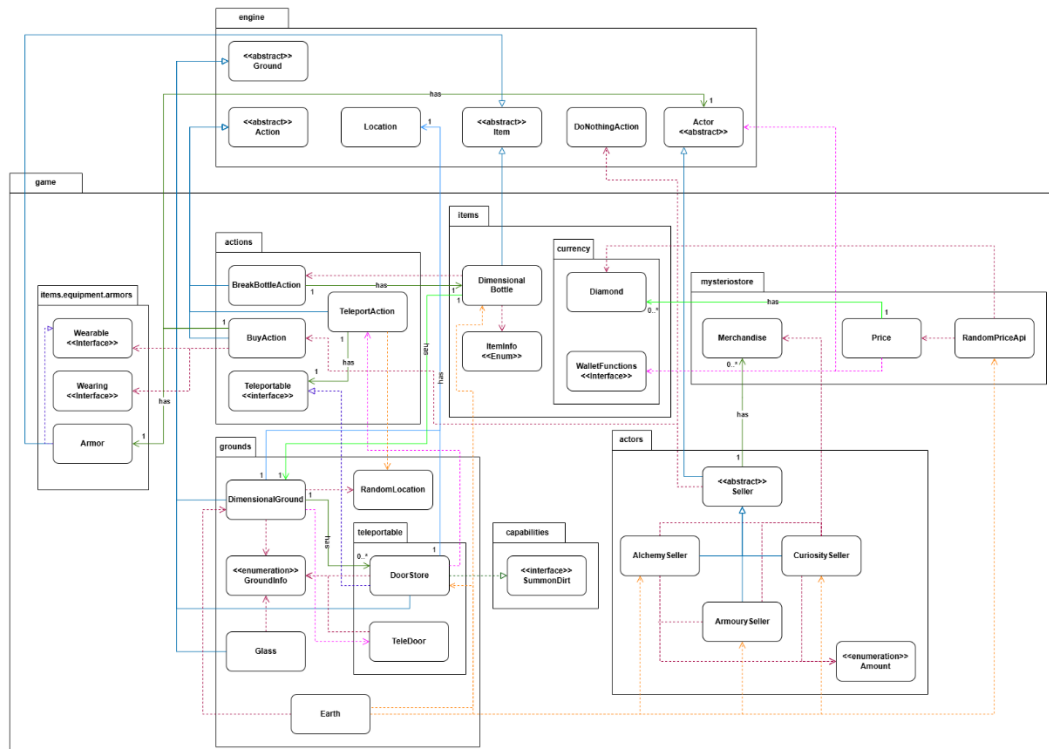
When a *Dimensional Bottle* is thrown, it transforms the ground into *Dimensional Ground*, which tracks its own tick count and, after three turns, activates a *Store Lifecycle* that randomly generates one of three *Mysterio Stores*. Each store hosts a distinct *Seller* (Armoury, Alchemy, Curiosity) that uses the RandomPriceApi to obtain dynamic prices from an external random-number service (with API-key authentication) and automatically falls back to local generation when the API is unreachable. The player can enter the store, buy items through a unified BuyAction, and the store automatically closes after three turns.

Pros	Cons
Adheres to SOLID principles: SRP: Each class focuses on one responsibility (e.g., DimensionalBottle handles spawning, Seller handles transactions). OCP: New store types or sellers can be added without modifying existing code. LSP / DIP: Interactions rely on shared abstractions (Seller interface, BuyAction) instead of concrete types.	The design introduces more classes and interfaces than a basic implementation, slightly increasing the learning curve for new developers.
DRY and maintainable: Common logic such as random pricing and item purchase is centralised in RandomPriceApi and BuyAction, eliminating duplication across multiple sellers.	The random API integration adds external dependency management (e.g., network availability, API key setup), which requires clear configuration instructions.
Simplified lifecycle management: All timing and store-opening logic are now handled directly inside DimensionalGround, removing the need for an extra lifecycle class. This keeps the control flow straightforward and easier to trace.	
KISS compliance: The new structure minimises class count and dependencies. Each component — DimensionalBottle, DimensionalGround, MysterioStore, and Seller — performs one clear task, ensuring simplicity without sacrificing functionality.	

Chosen Design:

Design 2 was chosen because it offers a balanced and maintainable structure that simplifies the overall system while keeping it fully object-oriented. By letting DimensionalGround manage store creation and closure directly, the design removes unnecessary lifecycle classes, reducing complexity and improving traceability. The use of clear interfaces such as Seller and reusable components like RandomPriceApi ensures low coupling and high cohesion. This approach follows the KISS, OOP, SOLID, and DRY principles—each class has a single, well-defined purpose, the code is easy to extend for new store types or sellers, and redundant logic is eliminated—making it the most

practical and scalable solution for the Mysterio Store system.



Existing Classes and Responsibilities

Seller (abstract) – Base class for all sellers; defines shared selling behaviour and price display.

ArmourySeller / CuriositySeller / AlchemySeller – Concrete sellers offering different items; use RandomPriceApi for dynamic pricing.

DimensionalBottle – Throwable item that creates DimensionalGround when broken.

BreakBottleAction – Executes the bottle-breaking and ground transformation.

DimensionalGround – Tracks turns and spawns a random Mysterio Store after three ticks, closing it after another three.

TeleDoor – Provides entry and exit between the store and original map.

Merchandise – Represents an item and its diamond price.

BuyAction – Handles the purchase process and applies the bought item automatically.

RandomPriceApi – Generates random prices via an external API, with local fallback.

Price – Stores item costs in green, blue, and red diamonds.

Earth – Integrates dimensional mechanics into the main map.

Classes Relate to and Interact with the Existing System

1. Map and Ground Relationship

DimensionalBottle and BreakBottleAction connect with the map system by transforming normal ground into DimensionalGround. The new ground type links to Earth and the tick system, ensuring temporal and spatial consistency within the existing environment.

2. Store and Seller Relationship

DimensionalGround interacts with the Seller hierarchy to spawn one of three store types — ArmourySeller, AlchemySeller, or CuriositySeller. Each seller manages its own Merchandise list, referencing Price objects and using RandomPriceApi for randomised diamond values.

3. Player Interaction Relationship

BuyAction bridges the Seller system and player inventory. It accesses Merchandise and Price to process transactions and applies item effects directly to the player. TeleDoor reconnects the store to Earth, maintaining logical consistency across maps.

4. API and Pricing Integration

RandomPriceApi links the in-game economy to external random-number services. It provides randomised diamond counts for Price, establishing an indirect connection between the store system and the player's currency management.

5. Item and Action Consistency

Merchandise, BuyAction, and ArmourySeller collectively integrate with the base Item hierarchy. This ensures that items sold in dimensional stores behave consistently with

those found elsewhere in the game, maintaining compatibility with existing pickup and equip systems.

Deliver Functionality

1. Dimensional Bottle Activation

Breaking a DimensionalBottle via BreakBottleAction converts normal ground into DimensionalGround, initiating the dimensional process on the existing map.

2. Store Generation

DimensionalGround tracks turns using tick(). After three turns, it randomly spawns one of three stores—Armoury, Alchemy, or Curiosity—each linked to a specific Seller.

3. Pricing and Merchandise

Each Seller lists items through Merchandise, with prices generated by RandomPriceApi. The system ensures unique, dynamic pricing with fallback support when offline.

4. Player Purchase

BuyAction handles transactions between the player and seller, deducting diamonds and applying items instantly, such as auto-equipping armour.

5. Store Closure

After three more turns, DimensionalGround closes the store and restores the terrain. TeleDoor then returns the player to a random point on the Earth map.

Conclusion

The final design of the Mysterio Store fits well into the existing game system. It lets the DimensionalGround control when the store appears and closes, which makes the process simple and easy to follow. The connection between Seller, BuyAction, Merchandise, and RandomPriceApi is clear, so the player can buy items smoothly and see random prices each time.

This design follows the ideas of KISS, OOP, SOLID, and DRY. Each class has one clear job, and the code is clean and easy to change later. New stores or sellers can be added without changing the old code, and the random price system has a backup plan if the API is not working. Overall, this design is simple, flexible, and stable, giving players a better and more interesting game experience.